OpenClaw Color Line Patrol

OpenClaw Color Line Patrol

1. Course Content
   - Course Overview
   - Learning Objectives
2. On-site Color Calibration (Pre-preparation)
   - 2.1 On-site Environment Graphical Color Calibration (`colorSelect`)
3. Preparation
4. Color Line Patrol Case Practice
   - Step 1: Issue Line Patrol Command
   - Step 2: Hand-Eye Downward View and Real-time Debug Window
5. Core Source Code Analysis
   - `FollowLine()` — Semantic Mapping and Independent Subprocess High-frequency Control

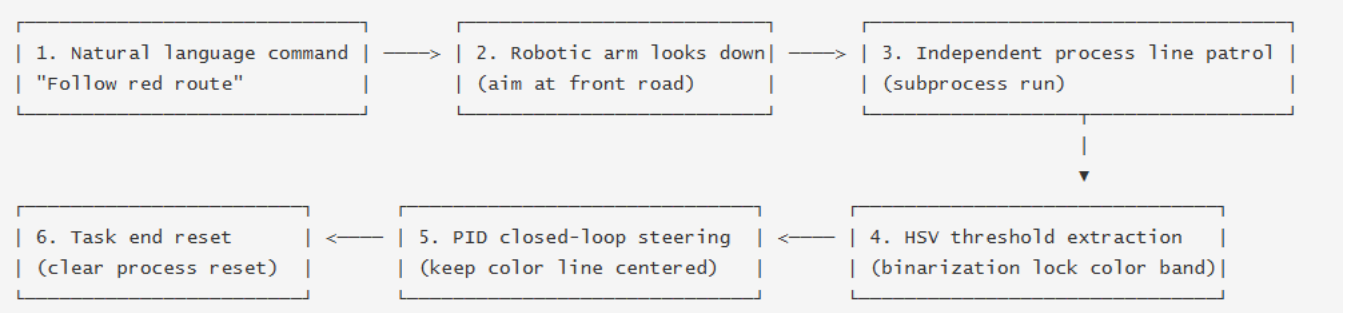
1. Course Content

Course Overview

In modern warehouse logistics AGV (Automated Guided Vehicle) and inspection robots, **visual line following (Visual Line Following)** is an autonomous tracking solution with extremely low cost and extremely flexible deployment. The robot only needs colored tape (such as red, green, blue, yellow tracks) laid on the ground, and it can travel smoothly along the established route.

In computer vision, there are two core keys to achieving highly robust line following:

1. **Why not use RGB, but must convert to HSV color space?**
   Ordinary RGB color mode tightly mixes "color" with "lighting brightness". The same red tape, when illuminated strongly, will appear pinkish white, and in shadows will appear dark grayish black, with vastly different RGB values, easily causing the car to get lost and "derail".
   However, **HSV space** (H Hue, S Saturation, V Value) completely separates "color's own attribute (Hue)" from "ambient brightness (Value)"! Even when lights are turned on/off or passing through sunlight-shadow boundaries, the red hue value almost remains unchanged, allowing the robot to stably and precisely extract the color line contour.
2. **High-frequency closed-loop and independent subprocess decoupling architecture**:
   Line following requires the chassis to calculate deviation and correct front wheel steering angle at extremely high frequencies above 30Hz. The system cleverly uses an independent subprocess to drive the algorithm, ensuring smooth line-following driving while not blocking the MCP large model's monitoring of other commands.



Learning Objectives

1. Skilled in using `colorSelect` graphical tool to complete on-site environment HSV threshold pre-calibration.
2. Master the method of starting red, green, blue, yellow four-color automatic line following through OpenClaw dialogue using `FollowLine` tool.
3. Understand HSV color space anti-interference mechanism for target extraction under complex ambient light.
4. Master multi-process management and master-slave asynchronous communication design philosophy.

---

2. On-site Color Calibration (Pre-preparation)

Before running the line patrol program, it is recommended to complete the color tape HSV calibration based on the on-site ambient lighting. Although the device has default calibration parameters from the factory, if your classroom/lab lighting is warm-toned, or the ground material reflections cause inaccurate recognition, please use the dedicated graphical tool for quick calibration.

- Color HSV threshold configuration file path:

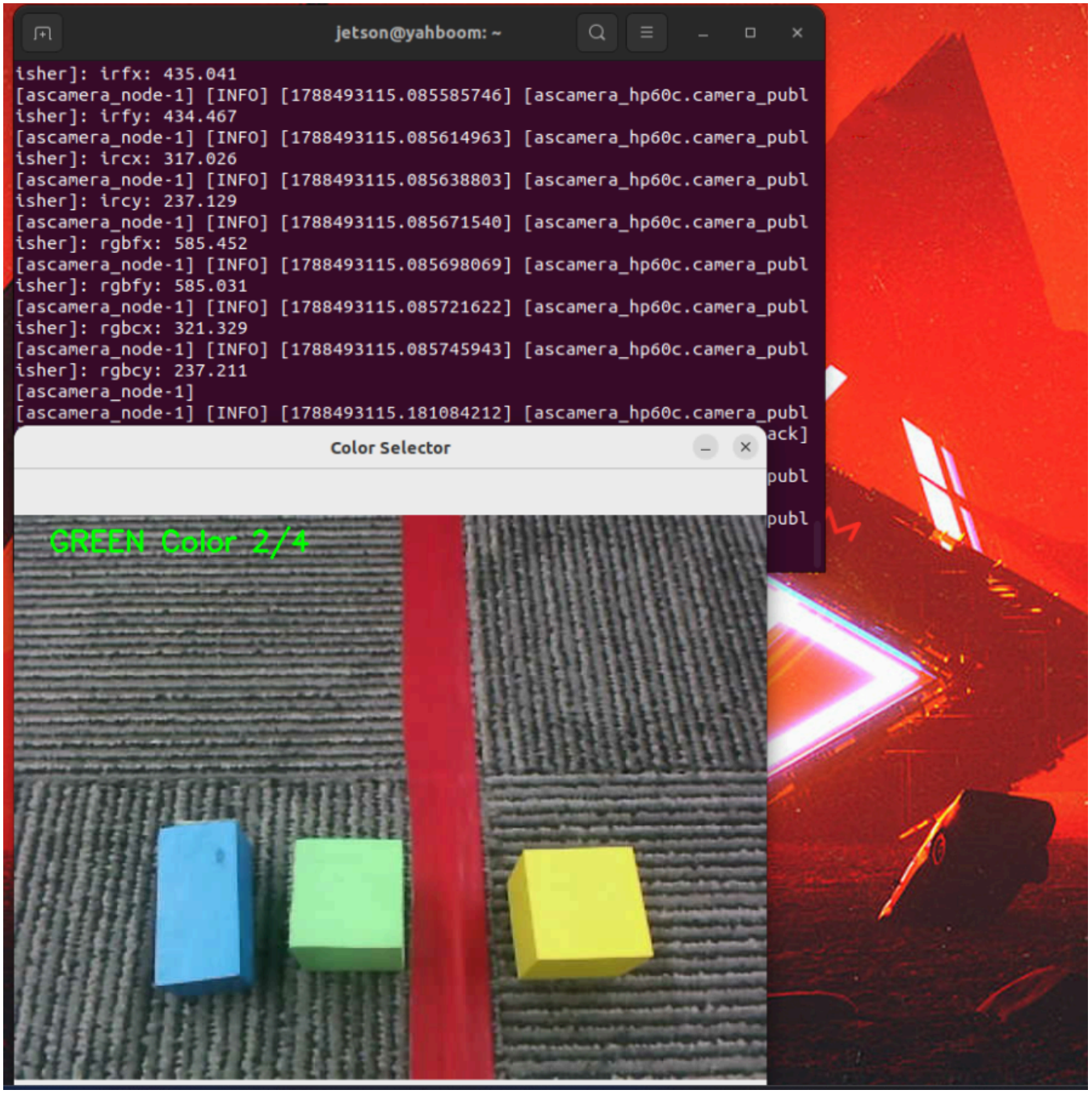
```
~/yahboomcar_ros2_ws/dofbot_pro_ws/src/ackermann_mcp/config/followlineHSV.text
```

2.1 On-site Environment Graphical Color Calibration (`colorSelect`)

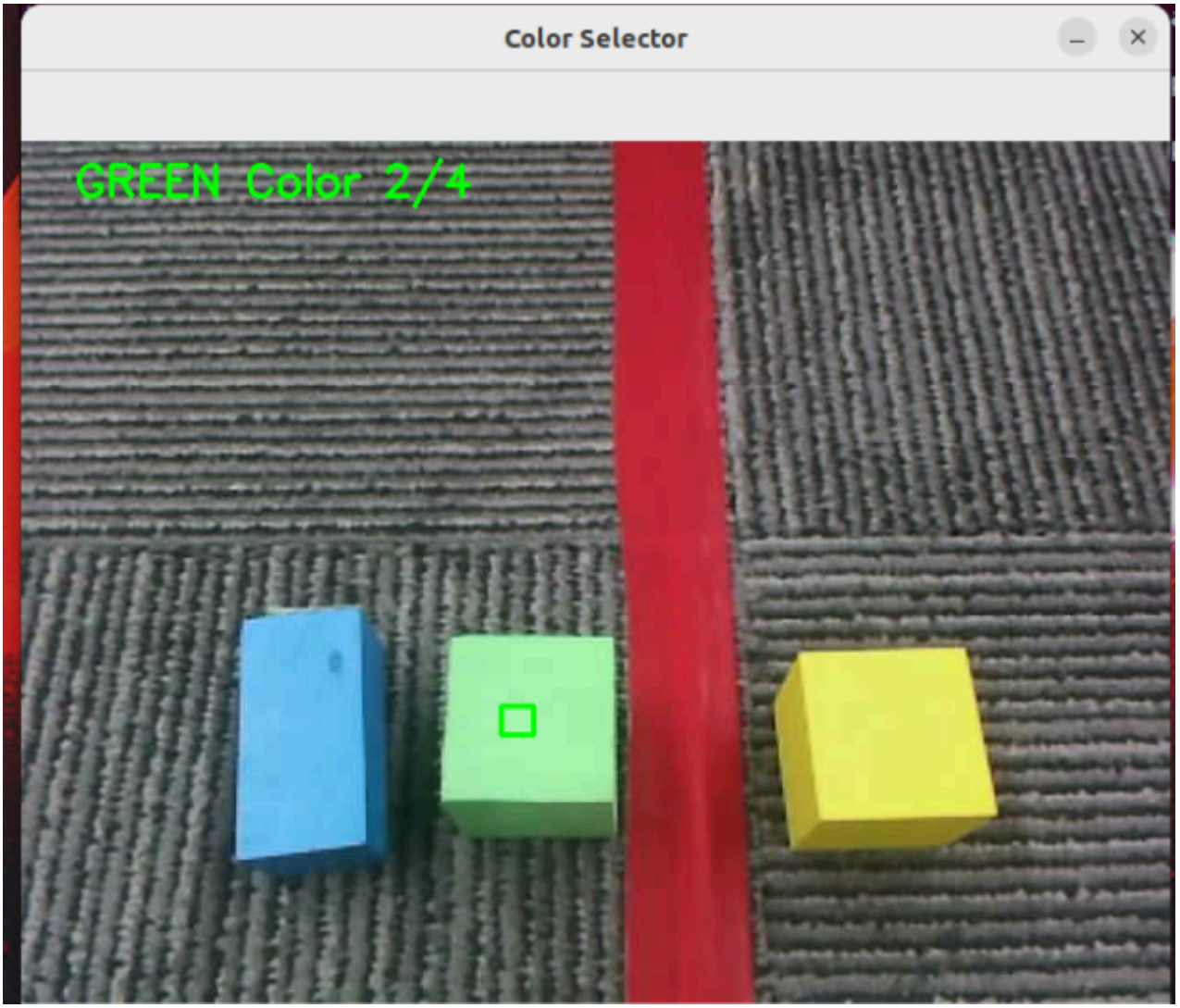
Enter the following command in the terminal to start the graphical calibration tool:

```
ros2 launch dofbot_pro_bringup colorSelect.launch.py
```

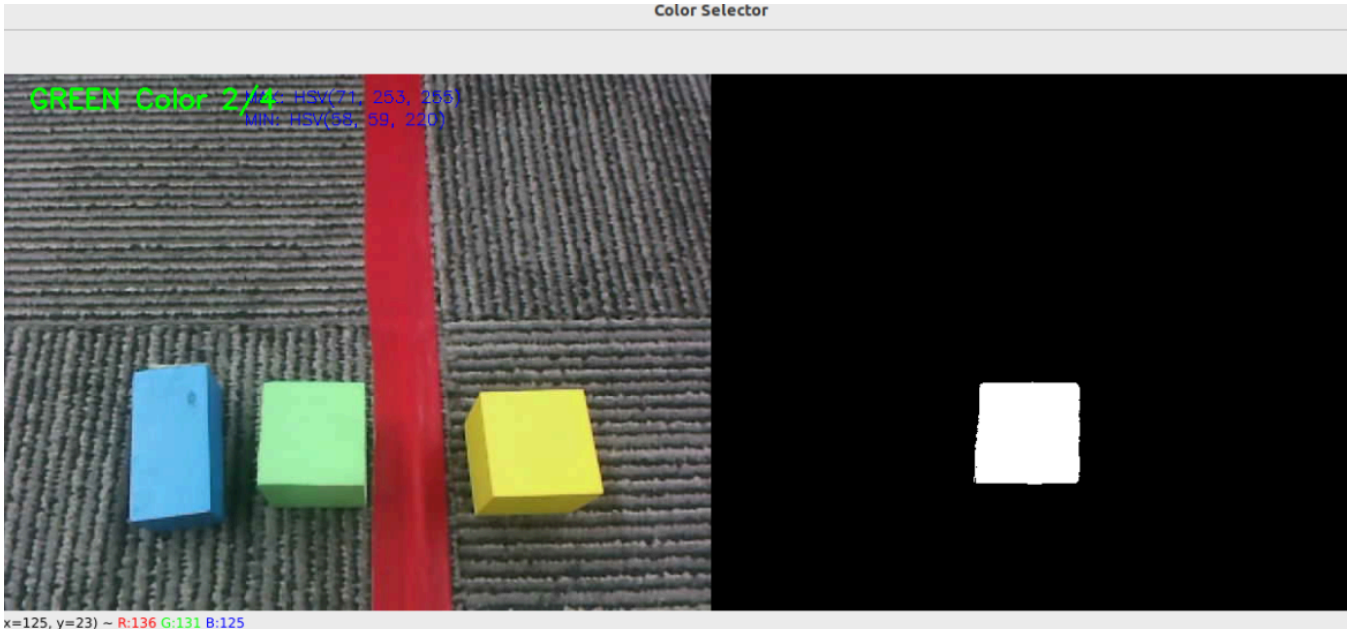
- After the program starts, a **Color Selector** interaction window will pop up on the screen:



- **Drag and select an area of pure color that needs calibration** in the window using the mouse (note: don't mix floor tile background colors into the selection):



- After selection, press the **Space key** on the keyboard, and the system will automatically calculate the maximum and minimum HSV range of the colors in the selection and persistently write to `followlineHSV.text`;
- After confirming the terminal prompt for successful saving, press `Ctrl + C` to exit the calibration program:



> **Note**
> After calibration saving is complete, be sure to first press `Ctrl + C` to close the `colorSelect` calibration node, then proceed with starting the line patrol main service, to avoid camera resource occupation conflicts.

---

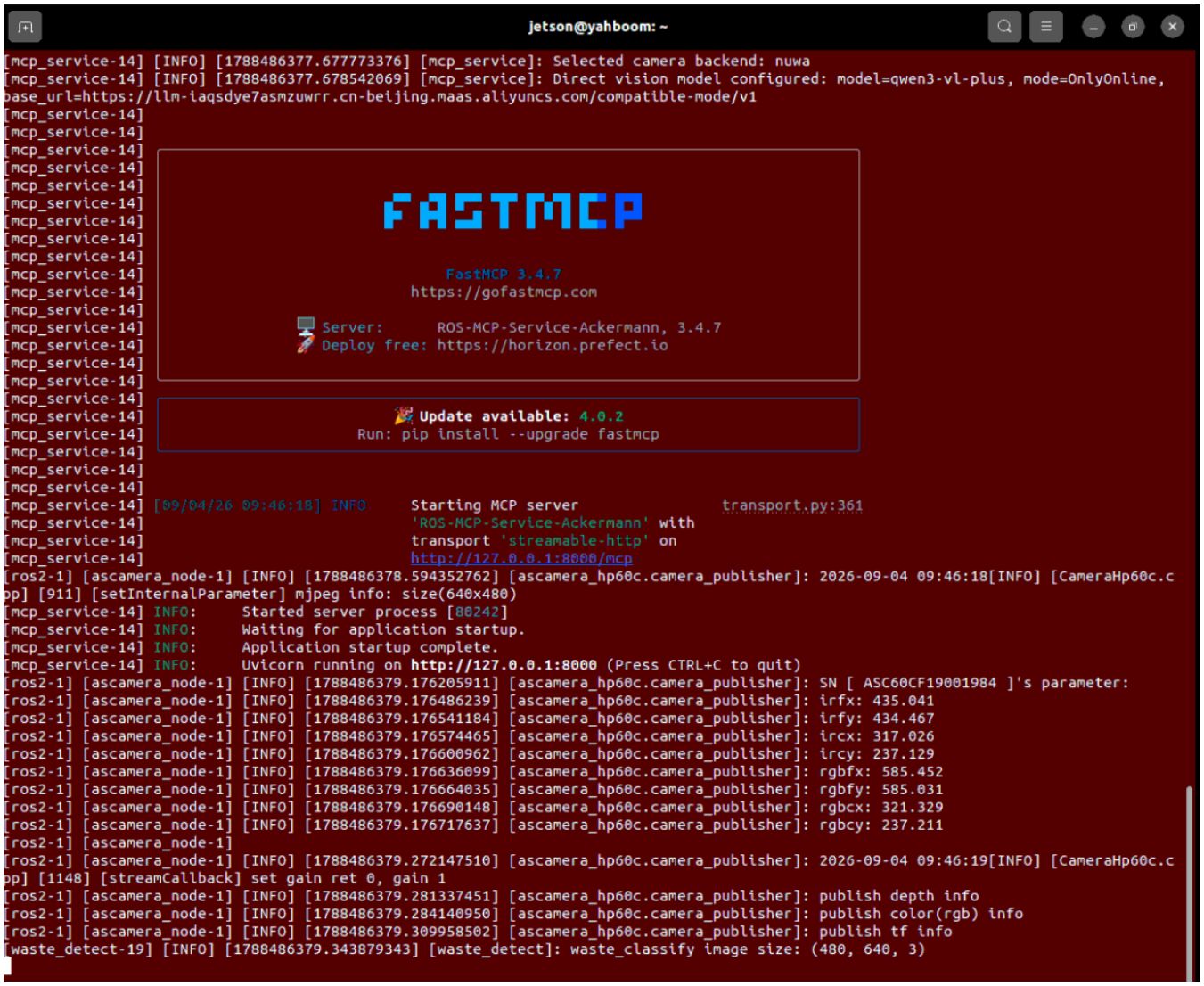
3. Preparation

After completing color calibration and exiting the calibration tool, start core nodes:

```
ros2 launch dofbot_pro_bringup mcp_service.launch.py
```

- If you need voice broadcast replies:

```
ros2 launch dofbot_pro_bringup mcp_service.launch.py   launch_openclaw_bridge:=true
```



- For voice dialogue interaction:

```
ros2 run ackermann_mcp asr_detect
```

```
jetson@yahboom:~/yahboomcar_ros2_ws/software/voice_toolbox_lib$ ros2 run ackermann_mcp asr_detect

ASR supplier: aliyun
[INFO] [1788487956.600357547] [asr_detect]: enable_wakeup_voice: True, enable_keybord_wakeup: False, en
able_kws_wakeup: True
[INFO] [1788487957.180189913] [asr_detect]:  ASR Engine: paraformer-realtime-v2 initialized successfull
y
[voice_toolbox] v1.0.0 loaded, copyright Yahboom Technology
[INFO] [1788487957.428767489] [asr_detect]: VAD engine initialized successfully
[kws_mic] Serial /dev/myspeech opened successfully
```

4. Color Line Patrol Case Practice

> **Tip**
> Instructions can be expressed naturally according to your actual needs. Here we use WebChat as an example.

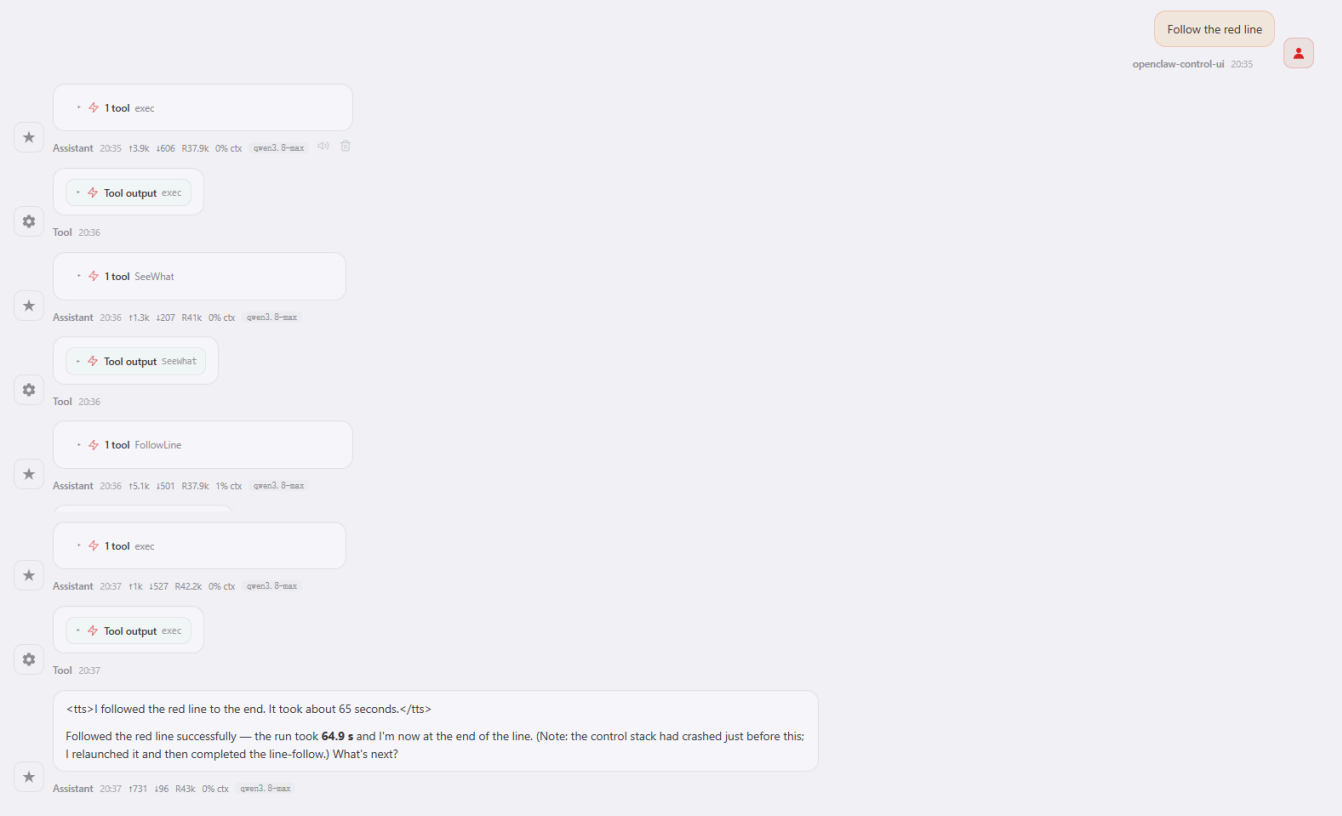
Core calling tool:

Tool Name	Input Parameters	Core Function
`FollowLine`	`--color` (supports: `red`, `green`, `blue`, `yellow`)	Start chassis to automatically follow the ground-specified color tape track for patrol driving

Step 1: Issue Line Patrol Command

Enter the line patrol command in the WebChat interface, for example:

> "**Follow the red line**" or "**Start green line patrol**"



Step 2: Hand-Eye Downward View and Real-time Debug Window

- The robotic arm quickly adjusts to the dedicated downward-looking posture, so the camera faces the road directly in front of the car;
- The system automatically pops up the line patrol monitoring window, with the left side showing RGB original color image, and the right side showing the **black and white binarized image** after HSV threshold filtering:



> **Image observation technique**:
> In the right binarization window, only the selected target color tape is displayed as pure white, all other background areas are pure black with no noise interference. The Ackermann chassis dynamically controls front wheel left-right steering based on the white area's geometric center deviation, guiding the car to travel stably as if on rails.

---

5. Core Source Code Analysis

- Source code file path:

```
~/yahboomcar_ros2_ws/dofbot_pro_ws/src/ackermann_mcp/ackermann_mcp/action_controller.py
```

`FollowLine()` — Semantic Mapping and Independent Subprocess High-frequency Control

```
def FollowLine(self, color: str) -> bool:
    '''Main entry for starting color line patrol task'''
    import subprocess
    # 1. Move robotic arm to preset observation posture, ensuring camera line of sight is
vertically downward to car front ground
    self._pubSix_Arm(self.waste_detect_pose)

    self.follow_line_future = Future()
    color = color.strip("'\"").lower()
```

```
    # 2. Map natural language color string understood by large model to the underlying
algorithm's agreed digital enum ID
    color_map = {'red': 1.0, 'green': 2.0, 'blue': 3.0, 'yellow': 4.0}
    if color not in color_map:
        error_msg = f"Unsupported color: '{color}'. Optional colors: red, green, blue,
yellow"
        self.get_logger().error(error_msg)
        return [False, error_msg]
    target_color = color_map[color]

    # 3. Core design: Start line patrol algorithm node as independent subprocess
(Subprocess)!
    self.follow_line_process = subprocess.Popen([
        'ros2', 'run', 'ackermann_mcp', 'follow_line',
        '--ros-args', '-p', f'color:={target_color}'
    ])

    # 4. Asynchronously listen for line patrol task end signal (triggered by chassis
detecting end transverse line or manual interruption)
    while not self.follow_line_future.done():
        time.sleep(0.1)
    self.follow_line_clear_future = None

    # 5. After task ends, thoroughly clean up subprocess tree, and restore robotic arm
initial posture
    kill_process_tree(self.follow_line_process.pid)
    self.InitArmPose()
    return [True, "Follow line finished"]
```

Key point popular explanation:

1. **Independent subprocess architecture advantages**:
   Image processing is extremely computationally intensive, and the line patrol algorithm needs to process images, calculate centroids and compute PID steering at more than 30 frames per second high refresh rate. If placed in the same thread as the main controller, once a tiny delay occurs, the entire large model MCP service may become unresponsive; using `subprocess.Popen` lets the line patrol exclusively have the process, being both fast and stable. The main program only nets at the end (cleans up process), and the architecture is very clean and robust.
2. **State graceful cleanup (`kill_process_tree`)**:
   After line patrol ends, the system uses the process tree cleanup function to thoroughly recycle all visual subprocesses, preventing zombie processes from remaining and consuming memory, while the robotic arm automatically and gracefully raises its head to reset.